

**Privacy Preserving Collaborative
Resource
and Task Orchestrator for Research
Environments
Software Requirements Specification
For The Complete System
Version 1.0**

Mentor: Mr. Dulaj Dilshan

Prepared by:

- 230328T Dinuka Kavinda
- 230324D P.M.M.C.S Karunarathna
- 230320L N.C.T Karunanayaka

Revision History

Date	Version	Description	Author
09/Aug/26	1.0	Initial Draft and Final Compilation of Software Requirements Specification	Group 12

Contribution

Member	Task
P.M.M.C.S Karunarathna 230324D	3.1 – Functionality 3.10 – Database Requirements 3.4 – Performance and Security 3.5 – Supportability
M.A.D Kavinda 230328T	1 – Introduction 2 – Overall Description 3.7 – On-line User Documentation and Help System Requirements 3.8 – Purchased Components 3.9 – Interfaces
N.C.T Karunanayaka 230320L	3.2 – Usability 3.3 – Reliability 3.11 – Licensing, Legal, Copyright, and Other Notices 3.12 – Applicable Standards 4 – Supporting Information

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and Dependencies
- 3. Specific Requirements
 - 3.1 Functionality
 - 3.1.1 User Authentication and Role Management
 - 3.1.2 Task and Project Orchestration (Kanban Board)
 - 3.1.3 Resource Scheduling and Booking
 - 3.1.4 Real-Time Collaboration and Communication
 - 3.1.5 Automated AI Context Engine
 - 3.1.6 Mobile On-The-Go Access
 - 3.2 Usability
 - 3.2.1 Intuitive Interfaces
 - 3.2.2 Role-Specific Views
 - 3.2.3 Error Handling and Feedback
 - 3.3 Reliability
 - 3.3.1 System Availability
 - 3.3.2 Graceful Degradation
 - 3.3.3 Mean Time Between Failures (MTBF)
 - 3.3.4 Mean Time To Repair (MTTR)
 - 3.3.5 Accuracy
 - 3.4 Performance and Security

- 3.4.1 End-to-End Encryption
 - 3.4.2 Real-Time Latency
 - 3.4.3 Database Locking Performance
 - 3.5 Supportability
 - 3.5.1 Microservices Architecture
 - 3.5.2 Containerization
 - 3.5.3 Version Control
 - 3.5.4 Testing and Quality Assurance
 - 3.5.5 Observability and Logging
 - 3.5.6 Coding Standards
 - 3.6 Design Constraints
 - 3.6.1 Self-Hostable Single-Tenant Constraint
 - 3.6.2 Localized AI Processing
 - 3.7 On-line User Documentation and Help System Requirements
 - 3.8 Purchased Components
 - 3.9 Interfaces
 - 3.9.1 User Interfaces
 - 3.9.2 Hardware Interfaces
 - 3.9.3 Software Interfaces
 - 3.9.4 Communications Interfaces
 - 3.10 Database Requirements
 - 3.11 Licensing, Legal, Copyright, and Other Notices
 - 3.12 Applicable Standards
- 4. Supporting Information
 - 4.1 References
 - 4.2 Appendices

Software Requirements Specification

1. Introduction

This Software Requirements Specification provides a comprehensive overview of the requirements for the "Privacy Preserving Collaborative Resource and Task Orchestrator for Research Environments." It captures the complete software requirements for the system, detailing the functional capabilities, performance goals, strict security mandates, and architectural constraints necessary to build a secure, self-hostable workspace for academic research teams.

1.1 Purpose

The purpose of this SRS is to fully describe the external behavior and internal constraints of the Privacy Preserving Collaborative Resource and Task Orchestrator. This document serves as the foundational blueprint for developers, designers, and testers throughout the project lifecycle. It details the specific functional requirements such as Kanban task tracking, resource booking, and AI-assisted meeting summaries as well as the critical non-functional requirements, with a heavy emphasis on end-to-end encryption (AES-256) and data privacy. It is intended for use by the core development team (Group 12), the project mentor, and university evaluators.

1.2 Scope

The software application being specified is a self-hostable, single-tenant SaaS platform designed specifically for academic research labs to coordinate their daily operations without risking data leaks. The system includes a React.js web application for desktop management, a React Native mobile companion app for on-the-go data capture and notifications, and a polyglot microservices backend. The system will facilitate interactive Kanban boards, shared lab resource scheduling, real-time collaboration, and an Automated Context Engine for chat summarization. Crucially, the scope enforces a strict "Privacy-First" architecture, ensuring all proprietary research data is encrypted at rest and in transit, isolating the lab's intellectual property from third-party cloud vulnerabilities.

1.3 Definitions, Acronyms, and Abbreviations

This table defines the technical terms, acronyms, and domain-specific vocabulary used throughout this document to ensure clear interpretation by all stakeholders.

Term / Acronym	Definition	Context in System
AES-256	Advanced Encryption Standard (256-bit key size)	Utilized in this project for strict end-to-end data encryption of all sensitive research payloads (at rest and in transit).

API	Application Programming Interface	Facilitates secure data communication between the React/React Native frontends and the core microservices.
GUI	Graphical User Interface	The visual web and mobile dashboards researchers use to interact with the task orchestrator.
JWT	JSON Web Token	Used for secure user authentication, role verification (RBAC), and stateless session management across the platform.
LLM	Large Language Model	The underlying AI models utilized securely via the localized LangChain framework to process chat data.
PI	Principal Investigator	The lead researcher or administrative head of the lab who manages project workspaces and approves AI-suggested tasks.
SaaS	Software as a Service	The software delivery model, implemented here specifically as a secure, self-hosted "Private SaaS" for isolated labs.
Single-Tenant	Single-Tenant Architecture	A setup where a single instance of the software and its supporting infrastructure serves only one specific research lab, preventing third-party data leaks.
SRS	Software Requirements Specification	This document, outlining all functional and non-functional requirements and constraints for the project.
UI/UX	User Interface / User Experience	The design principles focused on ensuring the system is intuitive, role-specific, and easy for scientists to adopt quickly.

1.4 References

- [1] Reuben LIMS, "Why Spreadsheets Struggle as Laboratories Grow," reubendigital.co.uk. [Online]. Available: <https://lims.reubendigital.co.uk/resources/blog/why-spreadsheets-struggle-as-laboratories-grow>. [Accessed: July 3, 2026].
- [2] React Native, "React Native: A framework for building native apps using React," reactnative.dev. [Online]. Available: <https://reactnative.dev/>. [Accessed: July 3, 2026].
- [3] LangChain AI, "LangChain Documentation," python.langchain.com. [Online]. Available: https://python.langchain.com/docs/get_started/introduction. [Accessed: July 3, 2026].
- [4] Atlassian, "What is Trello?," Trello. [Online]. Available: <https://trello.com/tour>. [Accessed: Jul. 3, 2026].
- [5] Slack Technologies, "What is Slack?," Slack. [Online]. Available: <https://slack.com/intl/en-gb/help/articles/115004071768-What-is-Slack>. [Accessed: Jul. 3, 2026].
- [6] Docker Inc., "Docker Documentation," docker.com. [Online]. Available: <https://docs.docker.com/>. [Accessed: July 11, 2026].
- [7] Spring, "Spring Boot Reference Documentation," spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. [Accessed: July 11, 2026].
- [8] Socket.IO, "Socket.IO Documentation," socket.io. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: July 11, 2026].
- [9] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001. [Online]. Available: <https://csrc.nist.gov/publications/detail/fips/197/final>. [Accessed: July 11, 2026].

1.5 Overview

The remainder of this SRS document is organized as follows: Section 2 provides an overall description of the system, outlining the product perspective, target user characteristics, and general constraints. Section 3 details the specific requirements, including comprehensive functional capabilities broken down by user roles, non-functional requirements such as usability, reliability, performance, and security (including encryption standards), as well as the specific hardware and software interfaces necessary for the microservices architecture. Section 4 contains supporting information and appendices.

2. Overall Description

This section provides a high-level background of the Privacy Preserving Collaborative Resource and Task Orchestrator. It describes the general factors that affect the product and its requirements, providing context for the specific technical requirements detailed in Section 3.

2.1 Product Perspective

Currently, academic research teams rely on a fragmented ecosystem of spreadsheets for resource management and third-party public cloud applications (such as standard Slack or Trello) for communication and task tracking. This introduces significant risks to intellectual property and

frequent administrative bottlenecks, such as double-booked lab equipment. The proposed system is a fully independent, self-contained Private SaaS (Single-Tenant) platform. It is not an add-on to existing software but a complete replacement tailored for academic labs. The system is distributed as a containerized Docker application, allowing research institutions to self-host the entire suite (Web App, Mobile App APIs, Database, and AI Engine) on their own isolated lab servers. This ensures complete data sovereignty and physically prevents third-party data leaks.

2.2 Product Functions

The platform provides a unified suite of tools designed to streamline laboratory administration:

- **Task and Project Orchestration:** Interactive, digital Kanban boards for tracking research tasks and milestones across different projects.
- **Resource Scheduling:** A centralized booking dashboard that utilizes database-level locking to prevent overlapping reservations of shared lab hardware, rooms, and software assets.
- **Real-Time Collaboration:** A secure messaging and rich-text editing environment for collaborative meeting minutes and lab documentation.
- **Mobile On-The-Go Access:** A mobile companion application providing instant push notifications and quick-capture capabilities for experimental notes taken directly at the lab bench.
- **Automated AI Context Engine:** A localized background service that processes chat interactions using Natural Language Processing to automatically generate meeting summaries, extract action items, and suggest task deadlines.

2.3 User Characteristics

The system is designed for four primary user roles within a research or academic institution. While users are generally highly educated and technologically literate, their primary focus is scientific research rather than software administration. Therefore, the UI must be highly intuitive to minimize the learning curve and reduce administrative friction.

User Role	Technical Expertise	Primary System Goals	Permissions Level
Institute Admin	High (IT Personnel / Sysadmins)	Deploying the self-hosted system via Docker, managing global security configurations, and	System-Wide Admin: Full access to infrastructure, global user provisioning, and system settings.

		overseeing cross-team resource availability.	
Principal Investigator (PI)	Moderate (Scientific Lead)	Assigning tasks, managing project permissions, approving AI-suggested tasks, and monitoring overall project progress and budgets.	Project Admin: Full control over their specifically created research workspaces and assigned team members.
Researcher / Team Member	Moderate (Lab Scientist)	Checking equipment availability, logging experimental notes from the lab bench via mobile, and communicating with peers in real-time.	Project Member: Read/Write access strictly isolated to their explicitly assigned projects and tasks.
Resource Manager	Moderate (Lab Operations)	Maintaining expensive laboratory equipment, setting scheduling parameters, and ensuring shared assets are utilized efficiently.	Resource Admin: Global access to the booking schedules, but explicitly restricted from viewing encrypted project data.

2.4 Constraints

The design and implementation of the system are limited by several critical constraints to ensure maximum security, data sovereignty, and the protection of intellectual property:

Req ID	Constraint Category	Description & Rationale	System Impact
SYS-CON-01	Security & Privacy	Strict "Privacy-First" Mandate: All sensitive payloads must be encrypted using AES-256 both at rest and in transit. <i>Rationale:</i> To ensure that highly sensitive research data is	Forces the system architecture to implement end-to-end encryption protocols across all client-server interactions before data hits the database.

		completely protected from unauthorized access, interception, or database breaches.	
SYS-CON-02	Architectural	Self-Hosted & Containerized: The system must not rely on managed multi-tenant cloud databases. All microservices and the PostgreSQL database must be fully containerized via Docker for isolated on-premise deployment. <i>Rationale:</i> To guarantee absolute data sovereignty for the adopting institution.	Prevents the use of standard managed cloud services (like shared AWS RDS), requiring robust local Docker Compose deployment configurations.
SYS-CON-03	AI Data Privacy	Localized AI Processing: The Automated Context Engine must run localized LLMs via LangChain. Sending data to public AI endpoints (e.g., standard ChatGPT) is strictly prohibited. <i>Rationale:</i> To protect proprietary intellectual property from being absorbed into third-party public AI training models.	Requires the integration of localized open-source LLMs or strictly secured, enterprise-tier APIs rather than basic public AI integrations.

2.5 Assumptions and Dependencies

- **Infrastructure Assumption:** It is assumed that the adopting research institution possesses the basic server hardware required to host local Docker containers (a minimum of 8GB RAM is recommended for prototype deployment).
- **Mobile Platform Dependency:** The mobile companion application assumes that researchers possess standard Android or iOS smartphones capable of receiving push notifications and running React Native applications.
- **Network Dependency:** While the system is self-hosted, a stable local area network or secure VPN is required for the web and mobile clients to maintain real-time WebSocket connections with the core server.

3. Specific Requirements

This section outlines the specific requirements of the system, detailing its intended behavior, core feature sets, and secure user interactions. These detailed specifications serve as the blueprint for designers to architect the platform and provide the exact criteria needed for testers to verify that the system operates efficiently and securely.

3.1 Functionality

This subsection details the functional capabilities of the Privacy Preserving Collaborative Resource and Task Orchestrator. The functional requirements are organized by major system modules and detail the specific user activities permitted within the system.

3.1.1 User Authentication and Role Management

This module governs how users access the system, verifies their identities, and enforces strict boundaries on what data they are permitted to view or modify based on their assigned roles.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-AUTH-01	Secure JWT Login	The system shall allow users to securely log in using email and password credentials, generating a secure JWT for session management.	High	All Users
FR-AUTH-02	Role-Based Access Control (RBAC)	The system shall enforce Role-Based Access Control accommodating four distinct roles: Institute Admin, Research Team Lead, Researcher/Team Member, and Resource Manager.	High	System
FR-AUTH-03	Global User Provisioning	The system shall allow the Institute Admin to provision new user accounts, assign roles, and configure global workspace security settings.	High	Institute Admin

FR-AUTH-04	Project Creation & Delegation	The system shall allow the Research Team Lead to create new research projects, add Researchers to the project, and define project-level access permissions.	High	PI / Team Lead
FR-AUTH-05	Strict Project Isolation	The system shall restrict the Researcher/Team Member role to read and write access strictly limited to the specific tasks, documents, and resources within their assigned projects.	High	Researcher
FR-AUTH-06	Resource Manager Restrictions	The system shall grant the Resource Manager access to manage global resource schedules (e.g., lab equipment, workstations) while explicitly restricting access to encrypted project data.	High	Resource Manager
FR-AUTH-07	E2E Key Exchange	The system shall perform end-to-end encryption key exchange during the authentication phase to ensure the user's device can decrypt their authorized project data.	High	System
FR-AUTH-08	Secure Logout	The system shall allow users to securely log out, which will instantly invalidate the active JWT session across both web and mobile platforms.	Medium	All Users
FR-AUTH-09	Password Recovery	The system shall provide a secure "Forgot Password" mechanism that verifies user identity via email before allowing a password reset.	Medium	All Users

FR-AUTH-10	Brute-Force Protection	The system shall deny access to unauthorized users and temporarily lock an account after a predefined number of consecutive failed login attempts to prevent brute-force attacks.	High	System
-------------------	-------------------------------	---	------	--------

3.1.2 Task and Project Orchestration (Kanban Board)

This module handles the core project management capabilities, allowing research teams to track their daily tasks, deadlines, and deliverables securely within isolated project workspaces.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-TASK-01	Project Kanban Boards	The system shall provide an interactive digital Kanban board for each created project.	High	PI / Researcher
FR-TASK-02	Task Creation & Assignment	The system shall allow authorized users, Research Team Leaders and Researchers to create new task cards, assign them to specific team members, and set due dates.	High	PI / Researcher
FR-TASK-03	Drag-and-Drop State Management	The system shall allow users to drag-and-drop task cards across customizable status columns (e.g., "To Do," "In Progress," "Under Review," "Completed").	High	PI / Researcher
FR-TASK-04	Rich-Text & Encrypted Attachments	The system shall allow users to attach encrypted files, insert rich-text descriptions, and add comments directly within individual task cards.	High	PI / Researcher

FR-TASK-05	Action-Based State Updates	The system shall update the Kanban board state upon a manual page refresh or immediately following a distinct user action (e.g., saving a comment, dropping a moved card).	Medium	System
FR-TASK-06	Asynchronous Task Alerts	The system shall generate system alerts or asynchronous notifications (e.g., via email or an in-app inbox) when a user is mentioned in a task comment or assigned to a new card, rather than relying on real-time push events.	Medium	System / All Users

3.1.3 Resource Scheduling and Booking

This module manages the cataloging, scheduling, and conflict-free booking of shared laboratory assets, utilizing strict database mechanisms to prevent overlapping reservations.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-RES-01	Centralized Resource Dashboard	The system shall provide a centralized Resource Dashboard displaying the availability of shared laboratory hardware, working rooms, and software assets.	High	All Users
FR-RES-02	Booking Management	The system shall allow users to view, manage, or cancel their active and upcoming resource bookings prior to the scheduled start time.	High	PI / Researcher

FR-RES-03	Booking Duration Limits	The system shall enforce maximum booking duration limits per user or project to ensure fair access to high-demand assets (e.g., specialized lab workstations or sensors).	Medium	System
FR-RES-04	Manager Override	The system shall allow the Resource Manager to manually approve, modify, or override any resource booking request in the event of lab emergencies or priority maintenance.	High	Resource Manager
FR-RES-05	Asset Catalog Management	The system shall allow the Resource Manager to add new assets to the catalog, define their maintenance schedules, and set booking parameters.	High	Resource Manager
FR-RES-06	Submit Booking Request	The system shall allow Researchers to select an available time slot and submit a booking request for a specific resource.	High	PI / Researcher
FR-RES-07	Concurrency & Conflict Prevention	The system shall utilize a strict database-level locking algorithm to instantly reject concurrent booking attempts for the same time slot, returning an "Already Booked" error to prevent overlaps.	Critical	System
FR-RES-08	Automated Maintenance Status	The system shall automatically update a resource's status to "Under Maintenance" during scheduled downtime, blocking all booking requests for that duration.	High	System

3.1.4 Real-Time Collaboration and Communication

This module facilitates secure, synchronous teamwork within isolated projects, ensuring researchers can communicate and document findings together without exposing unencrypted data over the network.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-COLLAB-01	Real-Time Project Chat	The system shall provide a secure, real-time messaging interface (Chat) within each project workspace for instant team communication.	High	PI / Researcher
FR-COLLAB-02	Collaborative Rich-Text Editor	The system shall provide a collaborative rich-text editor for documenting meeting minutes and experimental protocols.	High	PI / Researcher
FR-COLLAB-03	In-Transit Socket Encryption	The system shall ensure all real-time chat messages and collaborative keystrokes are encrypted before being transmitted through the Socket Server.	Critical	System
FR-COLLAB-04	AI Context Engine Trigger	The system shall provide a user interface option within the chat workspace and rich-text editor to send the current document or thread to the Automated Context Engine for processing.	Medium	PI / Researcher

3.1.5 Automated AI Context Engine

This module leverages localized Natural Language Processing (NLP) to reduce administrative overhead for researchers by automatically summarizing discussions and extracting action items, all while strictly confining data processing to secure, isolated environments.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-AI-01	Localized Context Engine	The system shall operate a Context Engine utilizing LangChain to process text data securely, supporting both self-hosted and SaaS deployment architectures.	High	System
FR-AI-02	Self-Hosted Data Sovereignty	For self-hosted deployments, the system shall process text data entirely within the client's private local infrastructure to maintain strict data sovereignty.	Critical	System
FR-AI-03	SaaS Tenant Isolation	For SaaS deployments, the system shall process text data strictly within the platform's secure, multi-tenant cloud architecture, ensuring tenant isolation.	High	System
FR-AI-04	Public Model Prohibition	The system shall explicitly prohibit the transmission of unencrypted research data to untrusted or public third-party AI models, regardless of the deployment environment.	Critical	System
FR-AI-05	Chat Summarization	The system shall scan project chat logs and generate concise, structured summary notes of team discussions.	High	System
FR-AI-06	Action Item Extraction (NLP)	The system shall use Natural Language Processing (NLP) to detect action-oriented phrases within chats and meeting minutes and extract them into discrete action items.	High	System

FR-AI-07	Automated Task Suggestion	The system shall convert extracted action items into suggested task cards and prompt the Research Team Leader or assigned Researcher for approval.	High	System
FR-AI-08	Task Review & Approval	The system shall allow users to review, edit, or reject AI-suggested task details prior to confirming their addition to the project Kanban board.	High	PI / Researcher
FR-AI-09	Manual AI Execution	The system shall allow authorized users to manually execute summarization and action extraction on selected chat history or documentation entries.	Medium	PI / Researcher
FR-AI-10	In-Memory Processing	The system shall process text strictly in-memory during context processing to prevent unencrypted research data from persisting on disk or leaking externally.	Critical	System

3.1.6 Mobile On-The-Go Access

This module extends the platform's core capabilities to smartphones, ensuring researchers can capture sensitive data securely and receive critical updates while actively working at the lab bench away from their main workstations.

Req ID	Requirement Name	Description	Priority	Target Role(s)
FR-MOB-01	Secure Mobile Companion App	The system shall provide a companion mobile application (iOS and Android) that securely connects to the platform's cloud-hosted SaaS backend services via standard HTTPS protocols.	High	PI / Researcher

FR-MOB-02	Quick-Capture Encrypted Notes	The system shall feature a "Quick-Capture" mobile interface allowing researchers to instantly type, encrypt, and upload short experimental notes directly from the lab bench.	High	PI / Researcher
FR-MOB-03	Real-Time Push Notifications	The system shall push real-time mobile notifications to a user's device for task updates, when they are mentioned in a chat, or when a resource booking status changes.	Medium	All Users
FR-MOB-04	Mobile Kanban View	The system shall allow users to view their assigned Kanban task cards and upcoming deadlines through a mobile-responsive interface.	High	PI / Researcher

3.2 Usability

This section details requirements ensuring the system is intuitive and easy to adopt for academic researchers and administrators.

3.2.1 Intuitive Interfaces

The web and mobile interfaces shall follow standard, recognizable design patterns (e.g., standard Kanban drag-and-drop mechanics) minimizing the learning curve.

Req ID	Usability Feature	Description & Target Metric	Validation Method
NFR-USE-01	Learning Curve	A new researcher must be able to create a task, post a chat message, and request a resource booking within 15 minutes of initial training.	User Acceptance Testing (UAT) with a sample group of new researchers.

3.2.2 Role-Specific Views

The system shall dynamically adjust the UI based on the user's role. For example, standard Researchers shall not see the complex server configuration panels available to the Institute Admin, reducing visual clutter and cognitive load.

Req ID	Usability Feature	Description & Target Metric	Validation Method
NFR-USE-02	Cognitive Load Reduction	Standard Researchers shall experience 100% obfuscation of the complex server configuration panels reserved for the Institute Admin.	UI/UX rendering tests across all four simulated user roles.

3.2.3 Error Handling and Feedback

The system shall provide clear, non-technical error messages. For instance, if a user attempts to book a time slot that was just taken, the system must immediately display an intuitive "Time Slot Already Booked" message rather than returning a raw database error code.

Req ID	Usability Feature	Description & Target Metric	Validation Method
NFR-USE-03	Clear Error Handling	100% of user-facing exceptions (e.g., concurrent booking conflicts) must resolve to intuitive UI alerts rather than exposing raw SQL/database error codes.	QA edge-case testing (triggering race conditions to verify UI response).

3.3 Reliability

This section outlines the expected stability, fault tolerance, and quantitative uptime metrics required for the system to reliably support daily laboratory operations.

3.3.1 System Availability

The core containerized services (PostgreSQL Database, Spring Boot API, and Node.js Socket Server) shall be designed to support 99.9% availability during standard laboratory working hours. System maintenance and Docker image updates shall be scheduled during off-peak hours to minimize operational disruption.

Req ID	Reliability Feature	Target Metric / SLA	Validation Method
NFR-REL-01	System Uptime	99.9% availability of core services during standard laboratory working hours.	Automated uptime monitoring and load testing via simulated traffic.

3.3.2 Graceful Degradation

The system shall employ graceful degradation to ensure that a localized failure does not result in a total system crash. Because the architecture relies on decoupled microservices, if a non-critical service (such as the Python AI Context Engine) fails or is taken offline, the primary Node.js chat server and React frontend must continue to function normally. Researchers must still be able to communicate manually, with AI summarization features temporarily disabled until the background service is restored.

Req ID	Reliability Feature	Target Metric / SLA	Validation Method
NFR-REL-02	Fault Tolerance	0% disruption to core Chat and Kanban UI if the isolated AI container crashes.	Chaos testing (intentionally terminating the AI container during active simulated chat sessions).

3.3.3 Mean Time Between Failures (MTBF)

The core system microservices shall achieve a Mean Time Between Failures (MTBF) of at least 720 hours (approximately one month of continuous operation) under standard user loads before requiring manual administrative intervention.

Req ID	Reliability Feature	Target Metric / SLA	Validation Method
NFR-REL-03	MTBF	> 720 hours of continuous operational stability under standard load.	Long-term endurance/stress testing in a staging environment.

3.3.4 Mean Time To Repair (MTTR)

Due to the containerized Docker architecture, the system shall maintain a Mean Time To Repair (MTTR) of less than 5 minutes for software-level failures. If a microservice container (e.g., the Node.js server) crashes unexpectedly, the Docker Engine's restart policies (via Docker Compose) must automatically detect the failure and spin up a healthy container instance without requiring a full server reboot.

Req ID	Reliability Feature	Target Metric / SLA	Validation Method
NFR-REL-04	MTTR	< 5 minutes to fully recover and restart crashed software containers.	Container failure injection monitoring via Docker health checks.

3.3.5 Accuracy

The system must maintain strict operational accuracy depending on the module. The centralized Resource Scheduling module must guarantee 100% accuracy in preventing concurrent double-bookings of laboratory assets, enforced via strict database-level locking. The Automated Context Engine shall maintain a minimum 85% accuracy rate in successfully identifying and extracting action-oriented tasks and deadlines from natural language chat logs.

Req ID	Reliability Feature	Target Metric / SLA	Validation Method
NFR-REL-05	Database Lock Accuracy	100% accuracy in preventing concurrent resource double-bookings.	Concurrent integration testing triggering simultaneous API requests.
NFR-REL-06	AI Extraction Accuracy	> 85% accuracy in successfully identifying action items from chat logs.	Manual QA audits comparing human-read transcripts to AI outputs.

3.4 Performance and Security

This section details the critical speed and data protection constraints.

3.4.1 End-to-End Encryption

To uphold the "Privacy-First" mandate, the system shall encrypt all sensitive payloads (chat messages, meeting notes, task descriptions) using the AES-256-GCM encryption standard at the application layer before they are stored in the PostgreSQL database or transmitted over the network.

Req ID	Security Feature	Target Metric / Standard	Validation Method
NFR-SEC-01	Application-Layer Encryption	100% of sensitive payloads encrypted via AES-256-GCM prior to database insertion or network transit.	Security auditing, packet sniffing (Wireshark), and database inspection to verify ciphertext.

3.4.2 Real-Time Latency

The Node.js Socket.io server shall process and broadcast real-time collaborative keystrokes (for meeting minutes) and chat messages with a maximum latency of 200 milliseconds to ensure a seamless, synchronous editing experience.

Req ID	Performance Feature	Target Metric / SLA	Validation Method
NFR-PERF-01	WebSocket Latency	< 200 milliseconds maximum latency for processing and broadcasting live keystrokes/chats.	Automated WebSocket load testing (e.g., using Artillery or JMeter) simulating multiple concurrent users.

3.4.3 Database Locking Performance

The Spring Boot Core API shall execute resource booking requests using strict database-level locking, ensuring that concurrent booking attempts are resolved and rejected within 500 milliseconds to prevent race conditions and overlapping schedules.

Req ID	Performance Feature	Target Metric / SLA	Validation Method
NFR-PERF-02	Transaction Resolution Speed	< 500 milliseconds to resolve, lock, and reject concurrent database booking attempts.	High-concurrency stress testing firing simultaneous API booking requests at the same time slot.

3.5 Supportability

This section indicates requirements that enhance the maintainability, testability, and long-term sustainability of the system for future development teams.

3.5.1 Microservices Architecture

The backend shall be implemented using a decoupled microservices architecture (Java Spring Boot, Node.js, Python FastAPI) to ensure that individual services can be updated, scaled, or debugged independently without requiring a complete system shutdown.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
NFR-SUP-01	Decoupled Architecture	100% decoupling of core services (Java, Node.js, Python) allowing independent restarts without total system failure.	Architectural review and isolated service-restart testing.

3.5.2 Containerization

All frontend assets, backend APIs, the AI engine, and the PostgreSQL database must be fully containerized using Docker and deployable via Docker Compose to ensure reproducible, consistent deployments across different laboratory servers.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
NFR-SUP-02	Container Deployment	100% of application components packaged into Docker containers and	Deployment verification on a fresh, isolated lab server environment.

		orchestratable via a single docker-compose command.	
--	--	---	--

3.5.3 Version Control

All source code, documentation, and Docker configuration files shall be maintained using Git, with repositories securely hosted (e.g., GitHub/GitLab) and protected by role-based access.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
NFR-SUP-03	Secure Versioning	100% of source code and configurations tracked via Git with strictly enforced Role-Based Access Control on the repository.	Access audits and commit history verification by the Institute Admin.

3.5.4 Testing and Quality Assurance

The system shall support automated unit testing for the Spring Boot API (using JUnit) and React frontends (using Jest) to ensure code integrity before deployment.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
NFR-SUP-04	Automated Testing	Integration of JUnit (Backend) and Jest (Frontend) test suites passing successfully prior to any production deployment.	Automated execution of test suites via a Continuous Integration (CI) pipeline.

3.5.5 Observability and Logging

The API Gateway and core microservices shall output standard structured logs (including timestamp, error code, and service name) to allow the Institute Admin to monitor system health and troubleshoot errors without exposing encrypted user data.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
--------	------------------------	--------------------------	-------------------

NFR-SUP-05	Privacy-Safe Logging	Standardized structured logging with 0% exposure of unencrypted user PII or proprietary research data in the log files.	Log parsing audits, manual log inspection, and simulated error generation.
-------------------	-----------------------------	--	--

3.5.6 Coding Standards

The development team shall adhere to standard language-specific naming conventions (e.g., CamelCase for Java/React, snake_case for Python/PostgreSQL) and utilize comprehensive inline code documentation to ensure the codebase remains maintainable for future researchers.

Req ID	Supportability Feature	Target Standard / Metric	Validation Method
NFR-SUP-06	Maintainable Codebase	Strict adherence to standard naming conventions (CamelCase/snake_case) accompanied by comprehensive inline documentation.	Static code analysis tools (e.g., SonarQube, ESLint) and manual peer code reviews.

3.6 Design Constraints

This section outlines mandated design decisions that strictly guide how the system is built and deployed to ensure security and data ownership.

3.6.1 Self-Hostable Single-Tenant Constraint

To guarantee absolute data sovereignty, the system must be designed as a single-tenant architecture. It must not rely on managed multi-tenant cloud databases (e.g., AWS RDS shared instances) but must be capable of being hosted entirely on an isolated, private laboratory server or a private cloud instance.

Req ID	Constraint Type	Description & Rationale	Validation Method
---------------	------------------------	------------------------------------	--------------------------

NFR-CON-01	Single-Tenant Deployment	Architecture MUST be single-tenant and self-hostable. <i>Rationale:</i> To guarantee absolute data sovereignty and prevent proprietary lab data from co-existing with other organizations on a shared cloud database.	Deployment and functional testing on a strictly air-gapped or localized private lab server.
-------------------	---------------------------------	---	---

3.6.2 Localized AI Processing

The Automated Context Engine must utilize LangChain to run localized or strictly secured LLM processes. Proprietary research chat logs must never be transmitted to public, unsecured AI training endpoints (e.g., standard ChatGPT web interfaces) to prevent intellectual property leaks.

Req ID	Constraint Type	Description & Rationale	Validation Method
NFR-CON-02	Localized LLM Execution	AI processing MUST occur via localized/secured LangChain endpoints. <i>Rationale:</i> To prevent proprietary research and intellectual property from leaking to or training public AI models.	Network traffic analysis (packet sniffing) to verify zero unauthorized outbound requests to public LLM endpoints during AI processing.

3.7 On-line User Documentation and Help System Requirements

The web application shall include an integrated "Help" section containing a user manual. This manual must provide step-by-step instructions for tasks such as creating project boards, booking resources, and configuring personal notification settings.

3.8 Purchased Components

The system heavily relies on open-source technologies (React, Spring Boot, PostgreSQL, Docker) to minimize costs. No major purchased components are currently mandated, aside from standard domain registration, SSL certification, and potential server hosting costs if the adopting institution chooses to deploy the Docker containers on a private cloud provider rather than local hardware.

3.9 Interfaces

3.9.1 User Interfaces

The system requires two primary user interfaces:

- **Web Dashboard:** Built with React.js, featuring a navigation sidebar, interactive drag-and-drop Kanban columns, a calendar-based resource booking view, and a collaborative rich-text editor pane.

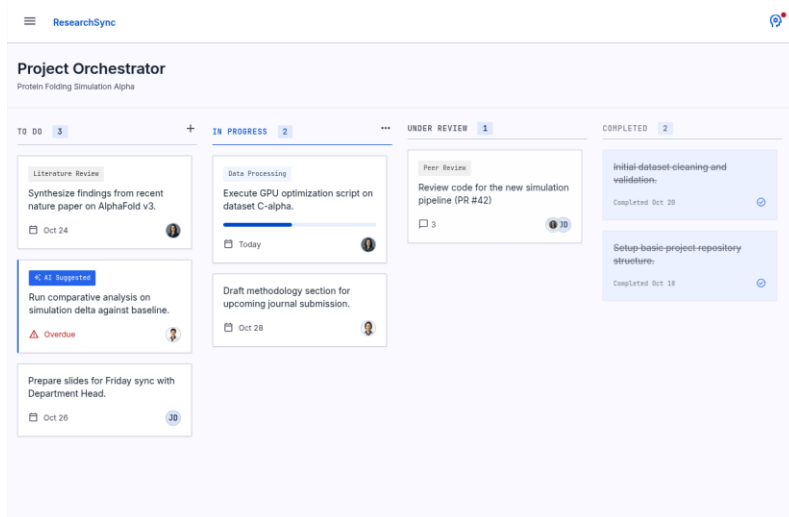


Figure A.1

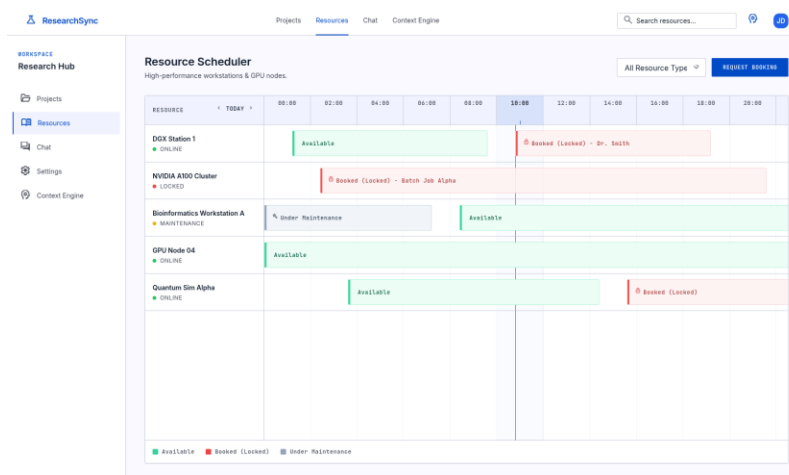


Figure A.2

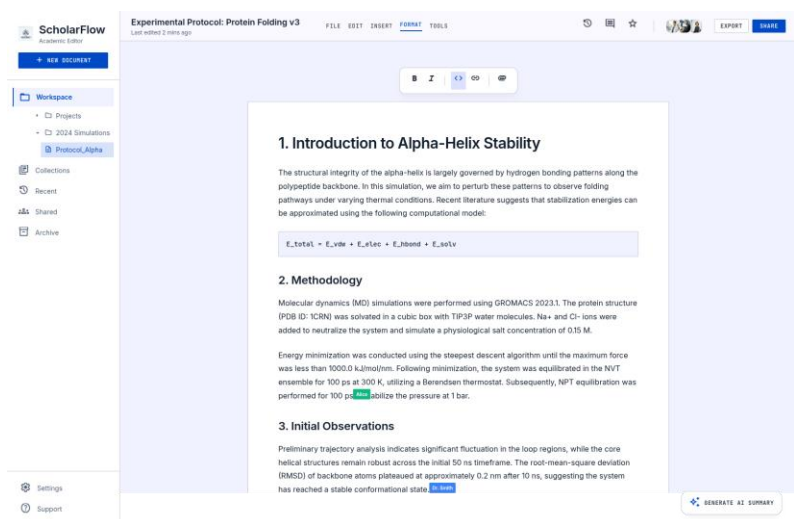


Figure A.3

- **Mobile Companion App:** Built with React Native, featuring a simplified dashboard for viewing assigned tasks, a quick-capture text input for experimental notes, and an interface for viewing push notifications.

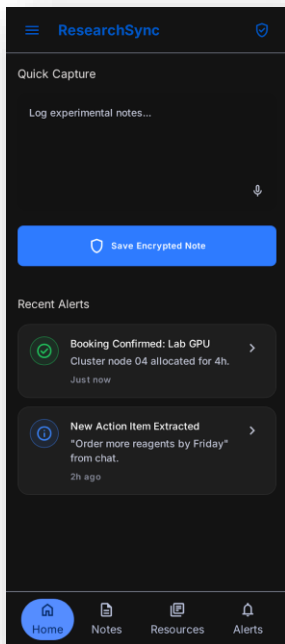


Figure A.4

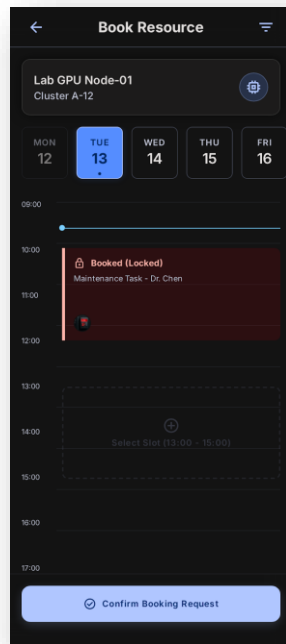


Figure A.5

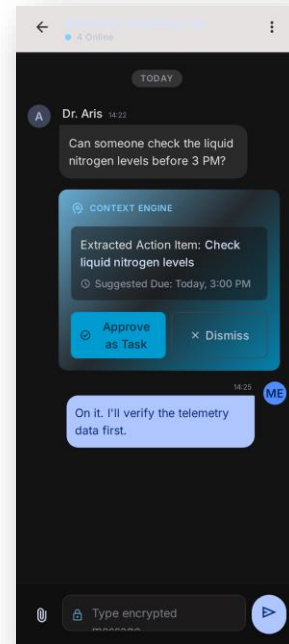


Figure A.6

Req ID	Interface Type	Target Component	Description / Technology
INT-UI-01	Web Dashboard	Desktop / Laptop Browser	React.js Single Page Application featuring Kanban boards, resource scheduling calendars, and rich-text editing panes.
INT-UI-02	Mobile App	iOS & Android Devices	React.js / React Native companion application for task tracking, quick-capture notes, and push notifications.

3.9.2 Hardware Interfaces

The self-hosted deployment requires a standard server environment (minimum 8GB RAM recommended for running local Docker containers concurrently). The mobile application requires users to possess standard iOS or Android smartphones.

Req ID	Interface Type	Hardware Requirement	Description
INT-HW-01	Server Environment	Private Lab Server / VM	Minimum 8GB RAM and multi-core CPU required to concurrently host Docker containers (PostgreSQL, Spring Boot, Node.js, Python AI).
INT-HW-02	Mobile Devices	Smartphones (iOS / Android)	Standard user smartphones capable of running React Native apps and receiving push notification services.

3.9.3 Software Interfaces

The system relies on internal REST APIs and WebSockets to communicate between the React frontends and the backend microservices. The Python AI Context Engine requires specific software interfaces (via LangChain) to process text arrays submitted by the Core API.

Req ID	Interface Type	Connected Services	Protocol / Mechanism
INT-SW-01	API Interface	React Web/Mobile to Spring Boot Core	JSON-formatted RESTful APIs for authentication, task management, and resource bookings.
INT-SW-02	Socket Interface	React Client to Node.js Server	WebSocket connections for real-time collaborative keystrokes and chat messaging.
INT-SW-03	AI Engine Interface	Spring Boot / Node.js to Python Service	Internal HTTP/RPC interface passing chat text payloads to LangChain for NLP processing.

3.9.4 Communications Interfaces

The system shall utilize standard HTTPS protocols for REST API requests and WSS for the real-time Node.js Socket.io connections, ensuring all transit data is encrypted.

Req ID	Interface Type	Transport Protocol	Security / Encryption
---------------	-----------------------	---------------------------	------------------------------

INT-COM-01	REST Communications	HTTPS (TLS 1.3)	Encrypted transport for all client-server API requests and responses.
INT-COM-02	Real-Time Communications	WSS (WebSocket Secure)	Encrypted bidirectional communications for live chat and document synchronization.

3.10 Database Requirements

The system shall utilize PostgreSQL as its primary relational database. The schema must enforce referential integrity across Users, Projects, Tasks, and Resources. The database must support transactional integrity (ACID properties) to successfully execute the required resource-locking algorithms during concurrent booking requests.

Req ID	Database Feature	Description & Rationale	Validation Method
DB-REQ-01	PostgreSQL RDBMS	The system MUST utilize PostgreSQL as its primary database. <i>Rationale:</i> To ensure a robust, open-source relational structure capable of handling complex scientific data relationships without licensing fees.	Infrastructure inspection and configuration review.
DB-REQ-02	Referential Integrity	The schema MUST enforce strict referential integrity. <i>Rationale:</i> To prevent orphaned records across core entities (Users, Projects, Tasks, and Resources) and maintain data consistency across the isolated workspaces.	Automated schema validation and integration testing targeting foreign key constraints (e.g., cascading deletes).

DB-REQ-03	ACID Transactional Integrity	The database MUST support strict ACID properties for transactional integrity. <i>Rationale:</i> Critical for successfully executing the resource-locking algorithms during concurrent lab equipment booking requests to prevent overlapping reservations.	Concurrent load testing and simulated race-condition transaction rollbacks.
------------------	-------------------------------------	--	---

3.11 Licensing, Legal, Copyright, and Other Notices

The system shall be developed in compliance with standard open-source licenses utilized by its core frameworks (e.g., MIT License for React, Apache License 2.0 for Spring Boot). Because it handles sensitive data, the deployment architecture must allow institutions to comply with local privacy laws and institutional data-sharing agreements (e.g., GDPR).

3.12 Applicable Standards

The encryption implementation must comply with standard cryptographic protocols, specifically AES-256 for data security. The development process shall adhere to standard RESTful API design principles.

4. Supporting Information

This section provides the references and supporting documentation necessary to interpret the technical requirements, design constraints, and algorithms defined within this Software Requirements Specification.

4.1 References

[1] Reuben LIMS, "Why Spreadsheets Struggle as Laboratories Grow," reubendigital.co.uk. [Online]. Available: <https://lims.reubendigital.co.uk/resources/blog/why-spreadsheets-struggle-as-laboratories-grow>. [Accessed: July 3, 2026].

- [2] React Native, "React Native: A framework for building native apps using React," reactnative.dev. [Online]. Available: <https://reactnative.dev/>. [Accessed: July 3, 2026].
- [3] LangChain AI, "LangChain Documentation," python.langchain.com. [Online]. Available: https://python.langchain.com/docs/get_started/introduction. [Accessed: July 3, 2026].
- [4] Atlassian, "What is Trello?," Trello. [Online]. Available: <https://trello.com/tour>. [Accessed: Jul. 3, 2026].
- [5] Slack Technologies, "What is Slack?," Slack. [Online]. Available: <https://slack.com/intl/en-gb/help/articles/115004071768-What-is-Slack>. [Accessed: Jul. 3, 2026].
- [6] Docker Inc., "Docker Documentation," docker.com. [Online]. Available: <https://docs.docker.com/>. [Accessed: July 11, 2026].
- [7] Spring, "Spring Boot Reference Documentation," spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. [Accessed: July 11, 2026].
- [8] Socket.IO, "Socket.IO Documentation," socket.io. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: July 11, 2026].
- [9] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001. [Online]. Available: <https://csrc.nist.gov/publications/detail/fips/197/final>. [Accessed: July 11, 2026].

4.2 Appendices

- **Figure A.1:** Web Application - Project Kanban Board & AI Context Engine Dashboard
- **Figure A.2:** Web Application - Shared Resource Scheduling Timeline
- **Figure A.3:** Web Application - Real-Time Collaborative Rich-Text Editor
- **Figure A.4:** Mobile Application - Quick-Capture Notes & Push Notifications
- **Figure A.5:** Mobile Application - Resource Availability & Booking
- **Figure A.6:** Mobile Application - Secure Project Discussion & AI Action Item Extraction